Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №7
Варіант 11
На тему
“functools та operator”

Виконав:
Студентка групи ФЕП-21
Муковоз М. Т.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета: освоєння функціональних інструментів Python (partial, reduce, singledispatch, total_ordering, засоби модуля operator) та вміння будувати функціональні конвеєри обробки даних.

Теоретичні відомості

Модуль functools надає інструменти для роботи з функціями вищого порядку. Функція partial дозволяє зафіксувати частину аргументів іншої функції, утворюючи нову з меншою кількістю параметрів. Функція reduce застосовує передану функцію до елементів послідовності (зліва направо), зводячи її до єдиного значення, що ідеально підходить для побудови конвеєрів (pipelines). Декоратор @singledispatch дозволяє створювати поліморфні функції, реалізація яких залежить від типу першого аргументу, що зручно для створення універсальних інтерфейсів (наприклад, для завантаження даних з різних джерел). Декоратор @total_ordering автоматично генерує відсутні методи порівняння (__le__, __gt__, __ge__) для класу, якщо в ньому реалізовані __eq__ та хоча б один інший метод порівняння (наприклад, __lt__).

Модуль operator містить ефективні реалізації вбудованих операцій Python у вигляді функцій. Функції itemgetter, attrgetter та methodcaller часто використовуються замість лямбда-виразів для вилучення елементів зі словників/кортежів, отримання атрибутів об'єктів або виклику їхніх методів відповідно. Їх використання робить код більш читабельним та іноді швидшим.

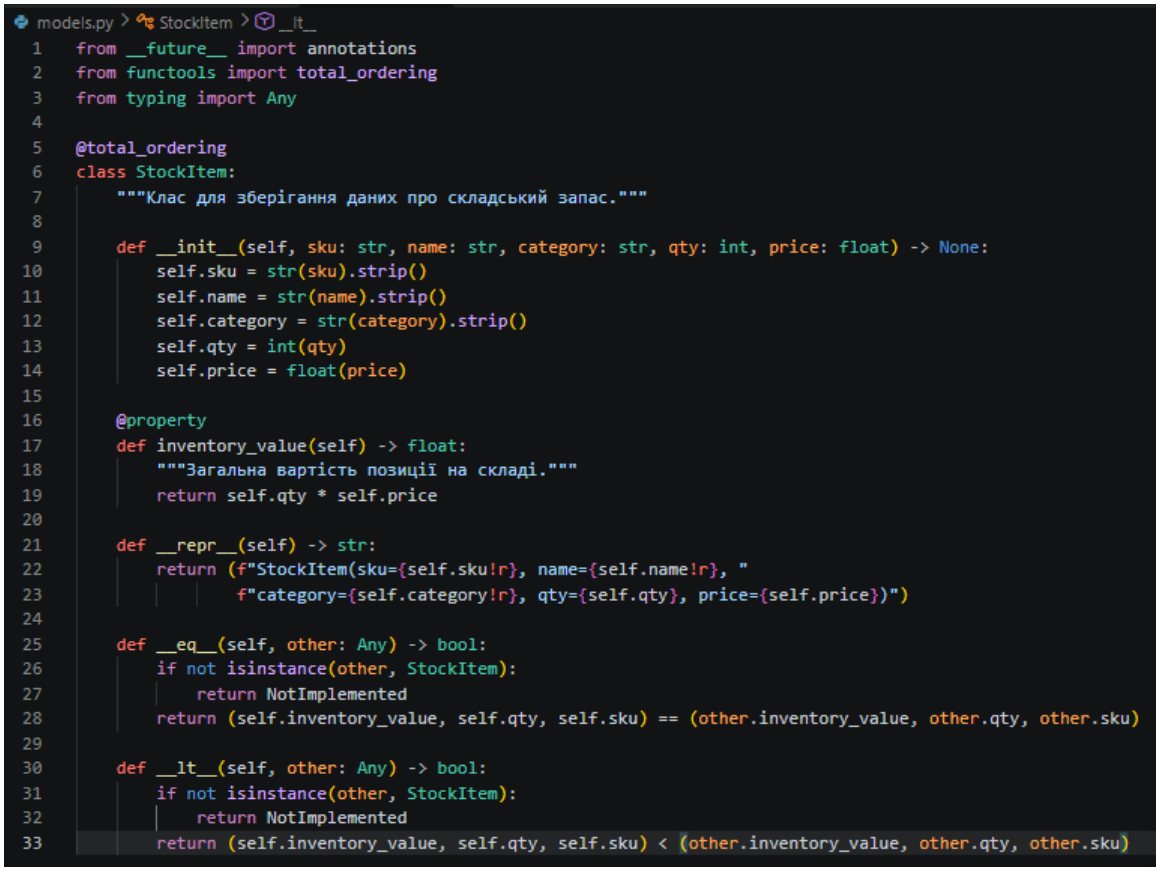
Хід роботи

Згідно з варіантом 11 (Складські запаси), було реалізовано функціональний конвеєр для обробки даних про товари на складі. Створено доменний клас StockItem з використанням декоратора @total_ordering для забезпечення повного набору операцій порівняння (за загальною вартістю запасів, кількістю, а потім за артикулом).

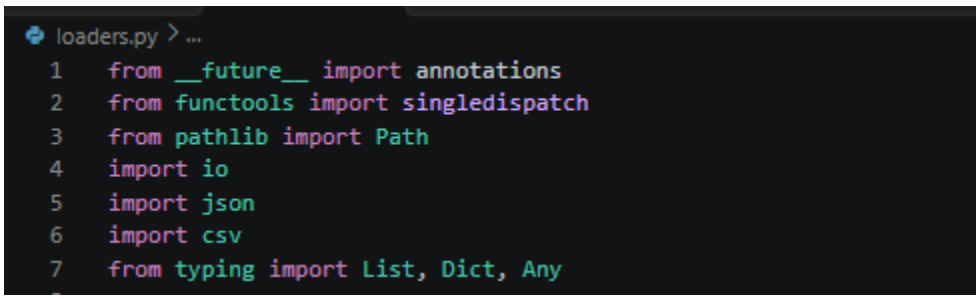
Реалізовано універсальну функцію load_inventory за допомогою @singledispatch, яка підтримує завантаження даних у форматах JSON

та CSV з рядків, об'єктів Path, відкритих файлів та списків словників. Створено конвеєр (pipeline) за допомогою functools.reduce, який послідовно застосовує завантаження, нормалізацію полів (з використанням operator.itemgetter та operator.methodcaller), перетворення у об'єкти StockItem, фільтрацію наявних товарів, сортування та вибірку Top-N результатів.

Код предметного класу StockItem (models.py):



Код поліморфного завантажувача (loaders.py):



```python
@singledispatch
def load_inventory(src: Any) -> List[Dict[str, Any]]:
    """Поліморфний завантажувач складських запасів."""
    raise TypeError(f"Unsupported source type: {type(src)!r}")

@load_inventory.register
def _(src: str) -> List[Dict[str, Any]]:
    s = src.strip()
    if not s:
        return []

    if s.startswith("[") or s.startswith("{"):
        data = json.loads(s)
        return data if isinstance(data, list) else [data]

    path_obj = Path(src)
    if path_obj.exists():
        content = path_obj.read_text(encoding="utf-8")
        if path_obj.suffix.lower() == ".csv":
            f = io.StringIO(content)
            reader = csv.DictReader(f)
            return list(reader)
        else:
            data = json.loads(content)
            return data if isinstance(data, list) else [data]

    if "," in s and "\n" in s:
        f = io.StringIO(s)
        reader = csv.DictReader(f)
        return list(reader)

    raise ValueError("Unrecognized string format for inventory data.")

@load_inventory.register
def _(src: Path) -> List[Dict[str, Any]]:
    content = src.read_text(encoding="utf-8")
    if src.suffix.lower() == ".csv":
        f = io.StringIO(content)
        reader = csv.DictReader(f)
        return list(reader)
    else:
        data = json.loads(content)
        return data if isinstance(data, list) else [data]
```

```python
@load_inventory.register
def _(src: io.TextIOBase) -> List[Dict[str, Any]]:
    content = src.read()
    content_stripped = content.strip()
    if content_stripped.startswith("[") or content_stripped.startswith("{"):
        data = json.loads(content)
        return data if isinstance(data, list) else [data]
    else:
        f = io.StringIO(content)
        reader = csv.DictReader(f)
        return list(reader)

@load_inventory.register
def _(src: list) -> List[Dict[str, Any]]:
    if src and not isinstance(src[0], dict):
        raise ValueError("Expected list of dictionaries for inventory items.")
    return src
```

Код конвеєра обробки даних (core.py):

core.py > ...

```python
from __future__ import annotations
from functools import reduce, partial
from operator import itemgetter, methodcaller
from typing import Callable, Iterable, Any, List, Dict

from models import StockItem
from loaders import load_inventory

def pipeline(value: Any, *funcs: Callable[[Any], Any]) -> Any:
    """Послідовне застосування функцій через reduce (конвеєр)."""
    return reduce(lambda acc, f: f(acc), funcs, value)

def mapf(fn: Callable[[Any], Any]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """Часткове застосування map для конвеєрів."""
    return partial(map, fn)

def filt(pred: Callable[[Any], bool]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """Часткове застосування filter для конвеєрів."""
    return partial(filter, pred)

def normalize_raw_item(row: Dict[str, Any]) -> Dict[str, Any]:
    """Нормалізує словник за допомогою operator."""
    get_sku = itemgetter("sku")
    get_name = itemgetter("name")
    get_cat = itemgetter("category")
    get_qty = itemgetter("qty")
    get_price = itemgetter("price")

    strip = methodcaller("strip")
    title = methodcaller("title")

    return {
        "sku": strip(str(get_sku(row))),
        "name": title(strip(str(get_name(row)))),
        "category": title(strip(str(get_cat(row)))),
        "qty": int(get_qty(row)),
        "price": float(get_price(row))
    }
```

```python

def item_to_object(d: Dict[str, Any]) -> StockItem:
    return StockItem(**d)

def top_n(n: int) -> Callable[[List[Any]], List[Any]]:
    """Повертає лямбду для зрізу перших n елементів."""
    return lambda it: list(it)[:n]

def clean_and_rank_inventory(src: Any, min_qty: int = 1, top: int = 3) -> List[StockItem]:
    """
    Головна функція для варіанту 11.
    Будує та виконує конвеєр обробки складських запасів.
    """
    is_available = lambda item: item.qty >= min_qty

    result = pipeline(
        src,
        load_inventory,
        mapf(normalize_raw_item),
        mapf(item_to_object),
        filt(is_available),
        list,
        partial(sorted, key=lambda s: (s.inventory_value, s.qty, s.sku), reverse=True),
        top_n(top)
    )
    return result
```

Демонстрація роботи програми (main.py) та результати виконання в консолі:

```python
from __future__ import annotations
from core import clean_and_rank_inventory
from models import StockItem

def main() -> None:
    print("ДЕМОНСТРАЦІЯ ЛАБОРАТОРНОЇ РОБОТИ №7 (Варіант 11: Складські запаси)")

    raw_json = (
        '[{"sku": "A-01", "name": "  laptop   ", "category": " electronics ", "qty": "5", "price": "1200.0"},'
        '{"sku": "A-02", "name": "mouse", "category": "electronics", "qty": "0", "price": "25.5"},'
        '{"sku": "B-01", "name": "office desk", "category": "furniture", "qty": "10", "price": "300.0"},'
        '{"sku": "C-01", "name": "ergonomic chair", "category": "furniture", "qty": "4", "price": "250.0"},'
        '{"sku": "A-03", "name": "usb-c cable", "category": "accessories", "qty": "50", "price": "15.0"}]'
    )

    print("\n1. Завантаження та ранжування складських запасів (Top 3 з qty >= 1):")
    top_items = clean_and_rank_inventory(raw_json, min_qty=1, top=3)
    for idx, item in enumerate(top_items, 1):
        print(f"  {idx}. {item} | Загальна вартість: {item.inventory_value}")

    print("\n2. Перевірка total_ordering та порівняння об'єктів:")
    item1 = StockItem("A-01", "Laptop", "Electronics", 5, 1200.0)
    item2 = StockItem("B-01", "Desk", "Furniture", 10, 300.0)
    print(f"  {item1} > {item2} -> {item1 > item2}")
    print(f"  {item1} <= {item2} -> {item1 <= item2}")

    print("\nДемонстрація успішно завершена!")

if __name__ == "__main__":
    main()
```

```
c:/Users/MTV/Desktop/3 sem/FunkProg/Lab_07/main.py"
ДЕМОНСТРАЦІЯ ЛАБОРАТОРНОЇ РОБОТИ №7 (Варіант 11: Складські запаси)

1. Завантаження та ранжування складських запасів (Top 3 з qty >= 1):
  1. StockItem(sku='A-01', name='Laptop', category='Electronics', qty=5, price=1200.0) | Загальна вартість:
6000.0
  2. StockItem(sku='B-01', name='Office Desk', category='Furniture', qty=10, price=300.0) | Загальна вартіст
ь: 3000.0
  3. StockItem(sku='C-01', name='Ergonomic Chair', category='Furniture', qty=4, price=250.0) | Загальна варт
ість: 1000.0

2. Перевірка total_ordering та порівняння об'єктів:
  StockItem(sku='A-01', name='Laptop', category='Electronics', qty=5, price=1200.0) > StockItem(sku='B-01',
name='Desk', category='Furniture', qty=10, price=300.0) -> True
  StockItem(sku='A-01', name='Laptop', category='Electronics', qty=5, price=1200.0) <= StockItem(sku='B-01',
 name='Desk', category='Furniture', qty=10, price=300.0) -> False

Демонстрація успішно завершена!
```

Успішне виконання тестів (pytest):

```
C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_07>python -m pytest
========================================= test session starts =========================================
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
benchmark: 5.3.0 (defaults: timer=time.perf_counter disable_gc=False min_rounds=5 min_time=0.000005 max_time
=1.0 calibration_precision=10 warmup=False warmup_iterations=100000)
rootdir: C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_07
configfile: pyproject.toml
plugins: benchmark-5.3.0
collected 5 items

tests\test_lab07.py .....                                                                    [100%]

========================================== 5 passed in 0.14s ==========================================
```

Висновок:

У ході виконання лабораторної роботи №7 за варіантом «Складські запаси» було успішно застосовано інструменти функціонального програмування з модулів functools та operator. Використання functools.reduce дозволило побудувати гнучкий та легко масштабований конвеєр обробки даних без застосування явних циклів. Завдяки functools.partial та інструментам operator (itemgetter, methodcaller) вдалося створити компактні та читабельні функції-перетворювачі. Декоратор @singledispatch забезпечив зручний поліморфний інтерфейс для завантаження даних із різних джерел та форматів (JSON, CSV), а @total_ordering значно спростив реалізацію багатокритеріального порівняння об'єктів предметної області. Написані автоматизовані тести підтвердили коректність роботи всіх розроблених компонентів конвеєра та функцій.